5 - Advanced Clustering Techniques

Giovanni Della Lunga
giovanni.dellalunga@unibo.it

Advanced Topics in Artificial Intelligence

Bologna - March, 11-28 2025

DBSCAN

Introduction to DBSCAN

- DBSCAN is a density-based clustering algorithm that identifies clusters as regions of high density separated by regions of low density.
- It can find arbitrarily shaped clusters and is robust to noise.
- Introduced by Martin Ester et al. in 1996.

Key Concepts

- **Epsilon (ε)**: The maximum distance between two points to be considered neighbors.
- **MinPts**: The minimum number of points required to form a dense region.
- **Core Point**: A point with at least MinPts neighbors within distance ε.
- **Border Point**: A point that is within ε of a core point but does not satisfy the MinPts condition.
- **Noise (Outlier)**: A point that is neither a core nor a border point.

Key Concepts

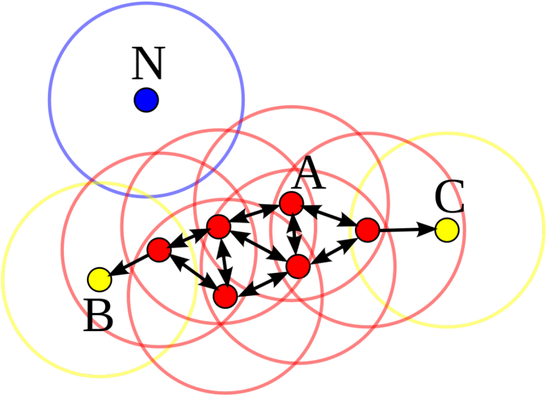


Figure 1: Point A and the other red points are core points, because the area surrounding these points in an ϵ radius contain at least 4 points (including the point itself). Points B and C are not core points, but are reachable from A (via other core points) and thus belong to the cluster as well. Point N is a noise point that is neither a core point nor directly-reachable

Mathematical Formulation

- A point p is directly density-reachable from q if:

$$d(p,q) \leq \varepsilon \quad \text{and} \quad |N_\varepsilon(p)| \geq \text{MinPts} \tag{1}$$

- Density-connectedness is defined as a transitive relationship over density-reachable points.
- A cluster is defined as a maximal set of density-connected points.

Advantages and Limitations

- **Advantages:**
  - Finds clusters of arbitrary shape.
  - No need to predefine the number of clusters.
  - Robust to noise.
- **Limitations:**
  - Performance degrades for high-dimensional data.
  - Sensitive to choice of ε and MinPts.

Application Examples

- **Geospatial Clustering:** Identifying clusters in geographical data (e.g., earthquake epicenters).
- **Anomaly Detection:** Fraud detection in financial transactions.
- **Image Segmentation:** Grouping similar pixels in image analysis.

Choosing ε and MinPts

- Use the k-distance graph to select an optimal ε.
- Rule of thumb: MinPts should be at least the number of dimensions plus one.

Conclusion

- DBSCAN is a powerful clustering method for spatial data and applications requiring noise detection.
- Choice of parameters is crucial for effective clustering.
- Suitable for various real-world applications from geospatial analysis to anomaly detection.

Spectral Clustering

Subsection 1

Introduction

What is Spectral Clustering?

Introduction

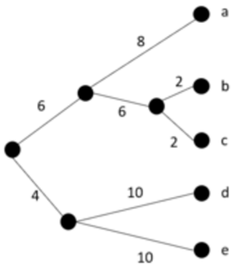
- A graph-based clustering method that uses eigenvalues of similarity matrices to find clusters.
- Unlike k-means, it captures non-convex and arbitrarily shaped clusters.
- Particularly effective when clusters have non-linear boundaries.
- Applications: Image segmentation, social network analysis, bioinformatics, recommendation systems.
- In the next slides, we briefly introduce the individual steps of the procedure and then move on to an in-depth analysis of each one...

Spectral Clustering Step by Step

Introduction

1. Similarity Matrix Creation: The first step involves creating a similarity matrix that *represents how similar each pair of points in the dataset is to each other*. This is often done using the Gaussian (RBF) kernel to convert the Euclidean distances between data points into similarity scores.

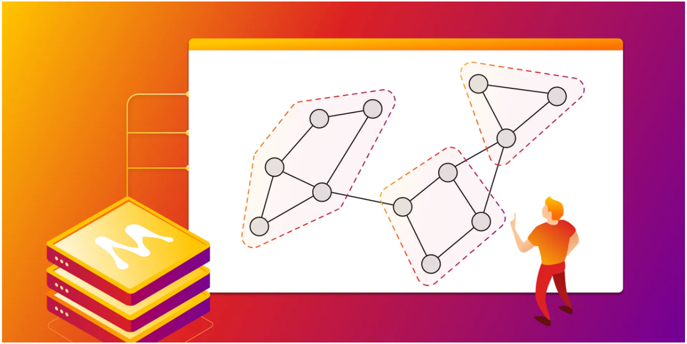
M	a	b	c	d	e
a	0	16	16	28	28
b	16	0	4	28	28
c	16	4	0	28	28
d	28	28	28	0	20
e	28	28	28	20	0



Spectral Clustering Step by Step

Introduction

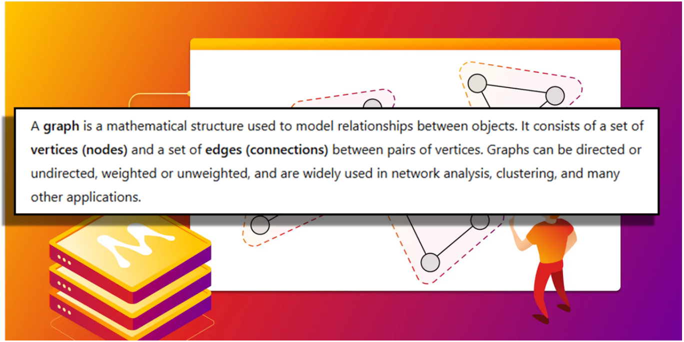
2. Graph Representation: The similarity matrix is then used to represent the data as a graph, with data points as nodes and the similarities as weights on the edges between nodes.



Spectral Clustering Step by Step

Introduction

2. Graph Representation: The similarity matrix is then used to represent the data as a graph, with data points as nodes and the similarities as weights on the edges between nodes.

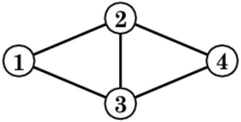


Spectral Clustering Step by Step

Introduction

3. Laplacian Matrix: From this graph, a Laplacian matrix is computed. The Laplacian is a matrix representation that captures the structure of the graph. It is used to find the minimum number of cuts needed to partition the graph into clusters.

$$D = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 2 \end{bmatrix}$$

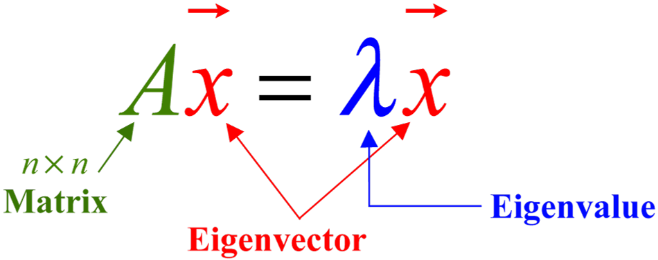


$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \qquad L = \begin{bmatrix} 2 & -1 & -1 & 0 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ 0 & -1 & -1 & 2 \end{bmatrix}$$

Spectral Clustering Step by Step

Introduction

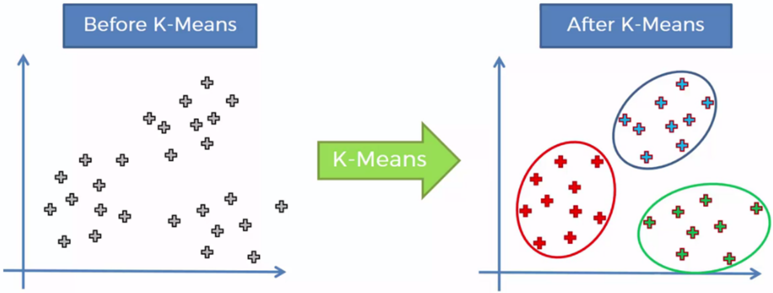
4. Eigenvalue Decomposition: The next step involves computing the eigenvalues and eigenvectors of the Laplacian matrix. The eigenvectors corresponding to the **smallest** eigenvalues (except for the smallest one, which is always zero) are used. These eigenvectors are then stacked to form a new set of features with reduced dimensions.



Spectral Clustering Step by Step

Introduction

5. Clustering: The reduced dataset is then clustered using a conventional algorithm like k-means. Since the data is now in a space where clusters are more distinguishable, k-means or similar algorithms can effectively identify the clusters.



Subsection 2

Mathematical Background

What is a Similarity Matrix?

Mathematical Background

The first step in spectral clustering is crucial as it lays the foundation for the entire clustering process. Let's delve into the details of this step:

- A **similarity matrix**, sometimes also referred to as an affinity matrix, is a *square matrix* used to represent the similarity between each pair of points in a dataset.
- In the context of spectral clustering, the dataset is typically composed of multidimensional points, and **the goal is to measure how close or similar these points are to one another**.
- The similarity matrix is **symmetric**, with its diagonal elements representing the similarity of each point with itself (usually the maximum similarity score) and off-diagonal elements representing the similarity between different points.

How is the Similarity Matrix Created?

Mathematical Background

- **Calculating Pairwise Distances**: The process begins by calculating the pairwise distances between all points in the dataset. This is often done using Euclidean distance, which measures the straight-line distance between two points in Euclidean space. However, other distance metrics can also be used depending on the nature of the data and the specific requirements of the clustering task.
- **Converting Distances to Similarities**: Directly using distances as measures of similarity is not intuitive because in a similarity context, we expect similar points to have a high score. Hence, the distances need to be converted into similarity scores. This conversion is typically achieved using a similarity function, such as the Gaussian (Radial Basis Function, RBF) kernel.

Using the Gaussian (RBF) Kernel

Mathematical Background

- The Gaussian kernel is a popular choice for transforming Euclidean distances into similarity scores. It is defined as follows:

$$S(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

  Here, $S(x, y)$ is the similarity between points x and y, $\|x - y\|$ is the Euclidean distance between x and y, and σ is a parameter that controls the width of the neighborhood or the spread of the Gaussian kernel.

- The effect of this transformation is that points closer to each other (smaller distances) result in higher similarity scores (closer to 1), while points further apart have lower scores (closer to 0).

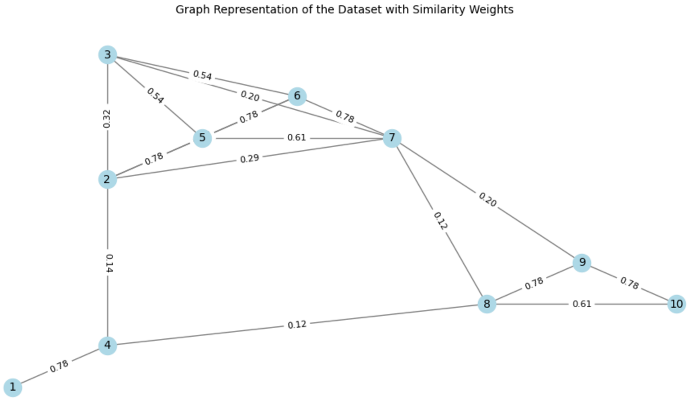
Key Considerations

Mathematical Background

- **Choice of σ**: The parameter σ plays a crucial role in determining the scale of similarity. Choosing an appropriate σ is essential for the success of the clustering.
  - A **small** σ makes the similarity drop off quickly with distance, leading to a sparser similarity matrix where only very close points are considered similar.
  - A **larger** σ makes the similarity drop off more slowly, considering a broader range of points as similar.
- **Sparsity of the Matrix**: In practice, for large datasets, the similarity matrix can be made sparse by setting all similarity scores below a certain threshold to zero. This reduces the computational complexity and focuses on stronger relationships, assuming that very distant points (low similarity) do not contribute significantly to the clustering structure.

Transform a Dataset into a Graph

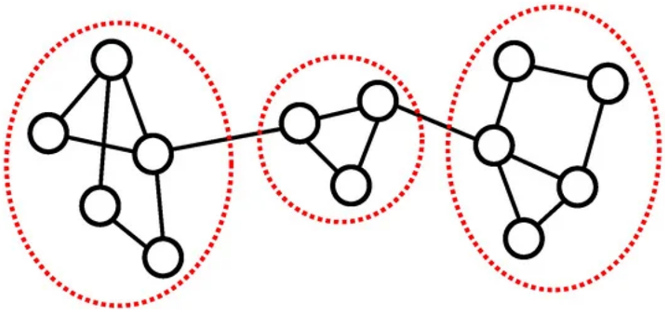
Mathematical Background



See the Jupyter Notebook **chapter-05-01.ipynb** for details and code

Transform a Dataset into a Graph

Mathematical Background



The intuition is that cutting the graph into subgraphs (clusters) can be done in such a way that **the connections (edges) between different clusters are weak (low similarity)**, while connections within a cluster are strong (high similarity)

Graph Laplacians

Mathematical Background

A quick recap of what we've done so far:

- Define similarity matrix $A \in \mathbb{R}^{n \times n}$ where A_{ij} represents the similarity between points i and j.
- Represent data as an undirected weighted graph $G = (V, E)$, where V is the set of data points and E represents weighted edges.
- The degree of each node is given by $D_{ii} = \sum_j A_{ij}$ (**sum of the weights of all edges connected to node** i), forming the diagonal degree matrix D.

Graph Laplacians

Mathematical Background

The **Laplacian matrix** is a crucial concept in graph theory and spectral clustering, in a nutshell it is a matrix representation that captures the graph's structure, focusing on how nodes (data points) are connected to each other via edges (similarities).

- The **unnormalized Laplacian** is defined as:

$$L = D - A \tag{2}$$

- The **normalized Laplacians** are given by:

$$L_{sym} = D^{-1/2} L D^{-1/2} = I - D^{-1/2} A D^{-1/2} \tag{3}$$

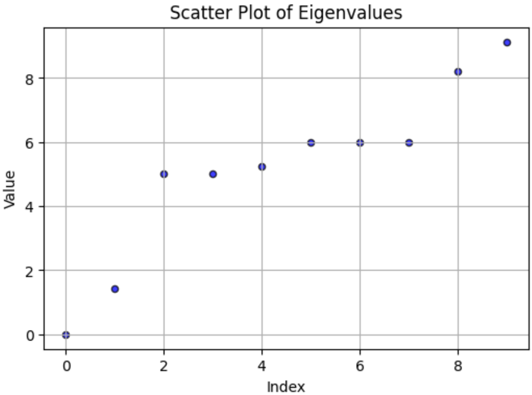
$$L_{rw} = D^{-1} L = I - D^{-1} A \tag{4}$$

- The spectrum (eigenvalues) of L reveals the structure of clusters.

Interpreting the Plot of Eigenvalues of the Laplacian

Mathematical Background

As we have said, the **eigenvalues of the Laplacian matrix** provide insights into the **structure** of a graph, including connectivity, clustering, and diffusion properties. When we plot them, we can extract valuable information about the dataset.



Interpreting the Plot of Eigenvalues of the Laplacian

Mathematical Background

1. **Eigenvalues and Graph Connectivity**
    - The smallest eigenvalue (λ_1) of the Laplacian **is always 0** for a connected graph.
    - The number of **zero eigenvalues** tells us how many **disconnected components** exist in the graph.
    - If **only one** eigenvalue is 0 $\rightarrow$ The graph is **fully connected**.
    - If there are **k zero eigenvalues** $\rightarrow$ The graph consists of **k disjoint clusters**.

Interpreting the Plot of Eigenvalues of the Laplacian

Mathematical Background

2. Spectral Gap and Cluster Structure

- The **second-smallest eigenvalue** (λ_2, the *Fiedler eigenvalue*) determines how **easily** the graph can be split into two clusters
  - **Small** $\lambda_2 \rightarrow$ Weakly connected graph $\rightarrow$ Easier to separate into two groups
  - **Large** $\lambda_2 \rightarrow$ Strongly connected graph $\rightarrow$ Harder to split.
- The **gaps between consecutive eigenvalues** help determine the **optimal number of clusters**.
- A **large jump** between λ_k and λ_{k+1} suggests that **k clusters** are a natural choice.

Choosing the Number of Clusters

- Analyze the **eigenvalue gap** (the spectral gap heuristic).
- Plot the sorted eigenvalues of L and locate the largest gap.
- The optimal k is determined by the index where the gap is largest.

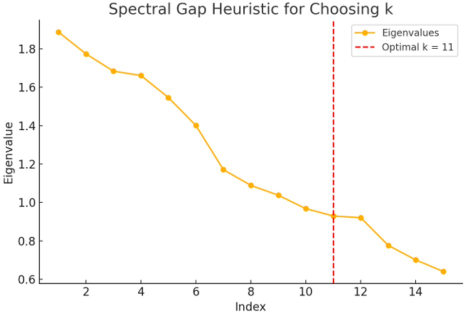


Figure 2: Spectral gap heuristic for determining k.

The Meaning of Eigenvectors in Spectral Clustering

Mathematical Background

Eigenvectors as Coordinates in a New Feature Space

- The original dataset is mapped into a new space defined by the eigenvectors of the Laplacian matrix.
- The first few non-trivial eigenvectors contain information about how nodes (data points) are connected.
- Clusters become more separable in this new space.

Spectral Clustering Algorithm

Algorithm 1 Spectral Clustering Algorithm

1: Construct the similarity graph and compute its adjacency matrix A.
2: Compute the degree matrix D.
3: Compute the graph Laplacian matrix L.
4: Compute the first k eigenvectors of L to form an embedding matrix U.
5: Normalize each row of U (for normalized Laplacian).
6: Apply k-means clustering to the rows of U.
7: Assign each point to a cluster based on k-means output.

Subsection 3

Comparison with Other Methods

Spectral Clustering vs. k-Means

Comparison with Other Methods

- k-means assumes convex clusters and uses Euclidean distance, which fails for complex structures.
- Spectral clustering leverages graph-based structures and does not assume convexity.
- Spectral clustering is computationally more expensive due to eigen decomposition ($O(n^3)$ complexity).

Spectral Clustering vs. Hierarchical Clustering

Comparison with Other Methods

- Spectral clustering is based on eigenvectors, whereas hierarchical methods use linkage criteria.
- Hierarchical clustering produces a dendrogram without requiring k upfront.
- Spectral clustering is typically more robust for high-dimensional data.

Spectral Clustering vs. Hierarchical Clustering

Comparison with Other Methods

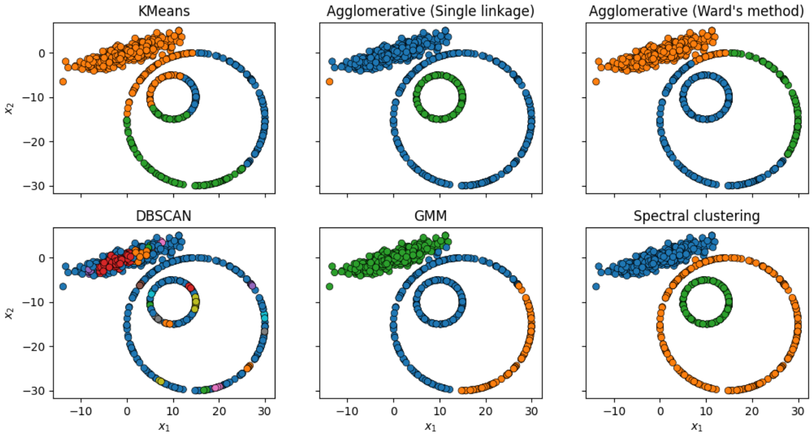


Figure 3: See the jupyter notebook **chapter-05-01.ipynb** for explanations

Limitations of Spectral Clustering

Comparison with Other Methods

- High computational cost due to eigen decomposition ($O(n^3)$ for large n).
- Performance depends heavily on the choice of similarity measure.
- Requires careful tuning of the similarity graph construction.

Subsection 4

Conclusion

Summary

- Spectral clustering leverages graph theory and eigenvalues to improve clustering.
- It captures complex, non-linear cluster structures better than traditional methods.
- While computationally expensive, it is useful for a variety of applications where traditional methods fail.

Gaussian Mixture Models (GMM)

Subsection 1

Definitions

Overview of GMM

- Gaussian Mixture Models (GMMs) are a type of unsupervised learning model that assumes all the data points are generated from a mixture of several Gaussian distributions with unknown parameters, each representing one cluster or component. These Gaussian distributions are blended together to form the complete dataset.
- GMMs are used for clustering similar to k-means but provide more flexibility due to the incorporation of distributional information. In particular, GMM provides a probabilistic clustering approach, allowing soft assignments.
- GMMs can accommodate clusters that have different sizes and correlations as well as different shapes, unlike k-means which assumes all clusters are spherical.
- Each Gaussian component is characterized by mean (μ), covariance matrix (Σ), and mixing coefficient (π).

Overview of GMM

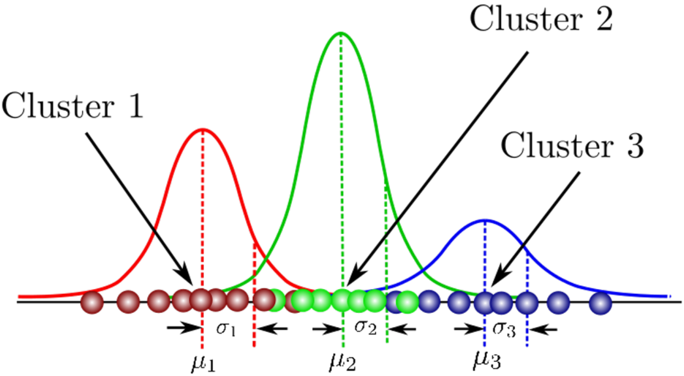


Figure 4: A simple example with three Gaussian functions, each Gaussian explains the data contained in each of the three clusters.

Overview of GMM

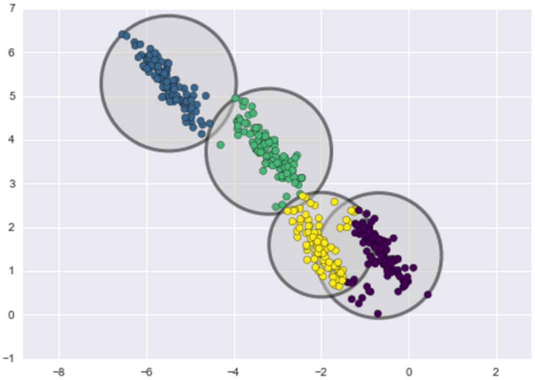


Figure 5: All the instances generated from a single Gaussian distribution form a cluster that typically looks like an ellipsoid. Each cluster can have a different ellipsoidal shape, size, density and orientation

Mathematical Formulation

- The probability density function (PDF) of a Gaussian mixture model is given by:

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x|\mu_k, \Sigma_k) \tag{5}$$

- Where:
  - π_k are the mixing coefficients such that $\sum \pi_k = 1$.
  - $\mathcal{N}(x|\mu_k, \Sigma_k)$ is the Gaussian distribution defined as:

$$\mathcal{N}(x|\mu_k, \Sigma_k) = \frac{1}{(2\pi)^{d/2}|\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_k)^T \Sigma_k^{-1}(x-\mu_k)\right) \tag{6}$$

  where d is the dimensionality of x.

Subsection 2

Expectation-Maximization Algorithm

Expectation-Maximization (EM) Algorithm

To estimate the parameters (means, covariances, and mixture weights) of the Gaussian components, GMMs typically use the **Expectation-Maximization (EM)** algorithm. The EM algorithm iteratively performs two steps until convergence:

- **Expectation Step (E-step)**: Given the current parameters, compute the probability that each data point belongs to each Gaussian component. This step essentially assigns weights to each data point based on how likely they are to belong to each component.
- **Maximization Step (M-step)**: Update the parameters of the Gaussians to maximize the likelihood of the data given these new assignments.

E-step: Computing Responsibilities

- The first step, known as the **Expectation step** or **E step**, consists of calculating the expectation of the component assignments C_k for each data point $x_i \in X$ given the model parameters π_k, μ_k and σ_k (the probability of each data point belonging to each distribution).
- For each point x_i compute r_{ic} the probability that x_i belong to the cluster c

$$r_{ic} = \frac{\pi_c \mathcal{N}(x_i \mid \mu_c, \Sigma_c)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)} \tag{7}$$

E-step: Computing Responsibilities

Initialization of Parameters

- In the Expectation-Maximization (EM) algorithm, the Expectation step (E-step) requires the posterior probabilities of each cluster given the data.
- However, in the **first iteration**, we don't yet have estimates for the parameters (μ_k, Σ_k, π_k).
- So, how do we start?

E-step: Computing Responsibilities

Initialization of Parameters

Since we don't know anything about the Gaussian distributions at the start, we need to **initialize** the parameters:

- **Means (μ_k)**: Usually initialized randomly or using K-Means clustering to get rough centroids.
- **Covariance matrices (Σ_k)**: Often set to identity matrices or computed from initial clusters.
- **Mixing coefficients (π_k)**: Set to **equal probabilities**, meaning each Gaussian starts with weight $\pi_k = \frac{1}{K}$.

After initialization, we proceed to the **first E-step**.

M-step: Parameter Update

Once we compute the responsibilities in the first **E-step**, we update the parameters in the **M-step**:

- Updating the means:

$$\mu_k = \frac{\sum_{i=1}^{N} r_{ik} x_i}{\sum_{i=1}^{N} r_{ik}} \tag{8}$$

- Updating the covariance matrices:

$$\Sigma_k = \frac{\sum_{i=1}^{N} r_{ik} (x_i - \mu_k)(x_i - \mu_k)^T}{\sum_{i=1}^{N} r_{ik}} \tag{9}$$

- Updating the mixing coefficients:

$$\pi_k = \frac{\sum_{i=1}^{N} r_{ik}}{N} \tag{10}$$

The parameters μ_k, Σ_k, and π_k are iteratively updated until convergence.

EM Algorithm

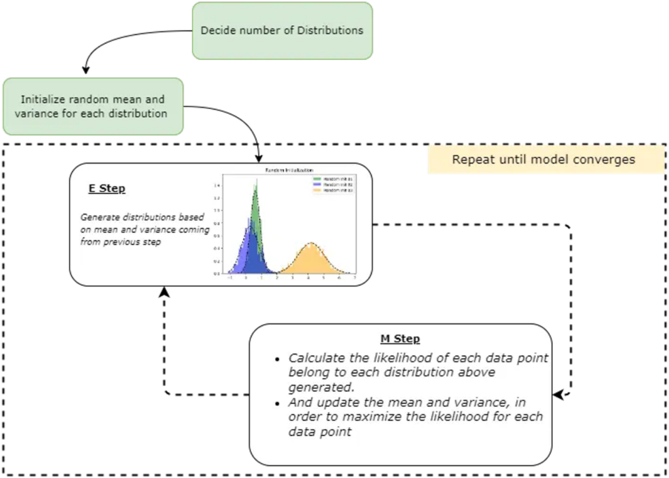


Figure 6: The Expectation-Maximization Process

Convergence of EM Algorithm

- The log-likelihood function is guaranteed to increase with each iteration.
- The algorithm stops when the change in log-likelihood is below a certain threshold:

$$|\log L^{(t)} - \log L^{(t-1)}| < \epsilon \tag{11}$$

- Convergence is usually fast, but EM can get stuck in local optima.
- Random initialization or multiple runs can help improve results.

Subsection 3

Advantages and Challenges of GMM

Advantages of GMM

- **Soft Clustering**: Unlike k-means, which assigns each data point to exactly one cluster, GMM provides probabilistic membership, allowing a more flexible clustering structure.
- **Modeling Complex Distributions:** GMM can model elliptical and overlapping clusters, which makes it superior to methods that assume spherical clusters, like k-means.
- **Handles Different Cluster Sizes and Shapes:** The covariance matrices allow clusters to have different orientations, densities, and scales.
- **Probabilistic Interpretation:** The output provides probabilities for each data point belonging to each cluster, which is useful in scenarios like anomaly detection and decision-making.
- **Adaptability to High-Dimensional Data:** GMM can work efficiently in high-dimensional feature spaces where simpler clustering algorithms may struggle.

Challenges of GMM

- **Sensitive to Initialization**: The algorithm's performance highly depends on initial parameter values. Poor initialization can lead to convergence to local optima.
- **Computational Complexity**: The Expectation-Maximization (EM) algorithm involves expensive matrix computations, particularly in high-dimensional spaces.
- **Prone to Overfitting**: If too many Gaussian components are used, GMM may fit noise in the data rather than underlying patterns.
- **Choosing the Number of Components**: Selecting the optimal number of Gaussians (K) is non-trivial and often requires model selection techniques like Bayesian Information Criterion (BIC) or Akaike Information Criterion (AIC).
- **Assumption of Gaussian Distributions**: GMM assumes that data is generated from a mixture of Gaussian distributions, which may not always be valid.

Mitigating Challenges

- **Improving Initialization**: Use k-means clustering or the K-means++ algorithm for better initialization of means before running EM.
- **Regularization**: Adding a small regularization term to covariance matrices can prevent singularities and overfitting.
- **Model Selection**: Use BIC and AIC to determine the optimal number of components, avoiding excessive model complexity.
- **Multiple Runs**: Running EM multiple times with different initializations can help avoid poor local optima.
- **Dimensionality Reduction**: Applying PCA or other dimensionality reduction techniques can help alleviate computational burdens in high-dimensional data.

Subsection 4

Selection and Evaluation of GMM

Choosing the Number of Components

- The number of Gaussian components (K) in GMM significantly affects performance.
- Common methods to determine K:
  - **Bayesian Information Criterion (BIC)**: Penalizes model complexity to avoid overfitting.
  - **Akaike Information Criterion (AIC)**: Measures model fit but does not penalize complexity as strongly as BIC.
  - **Cross-validation**: Splitting data into training and validation sets to evaluate model generalization.
  - **Elbow Method**: Observing the log-likelihood curve and selecting the point where improvement slows.

Choosing the Number of Components

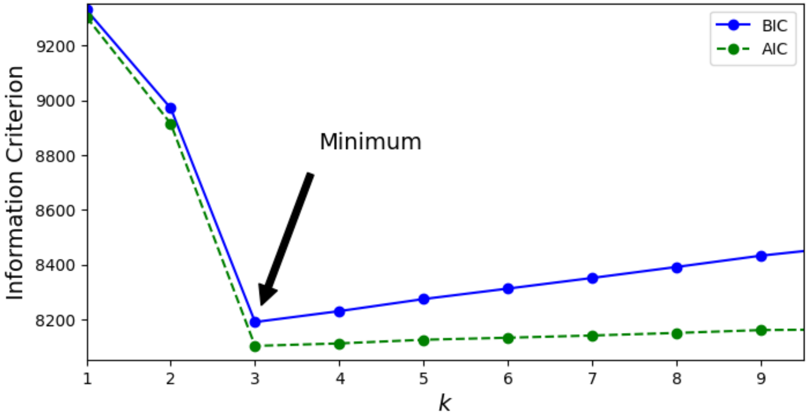


Figure 7: The application of the **Information Criterion** to find the optimal number of components

Evaluating GMM Performance

- **Log-Likelihood**: Measures how well the model explains the observed data.

$$L(\Theta) = \sum_{i=1}^{N} \log \sum_{k=1}^{K} \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k) \tag{12}$$

- **BIC and AIC Comparison**:

$$\mathrm{BIC} = -2 \log L + K \log N, \quad \mathrm{AIC} = -2 \log L + 2K \tag{13}$$

- **Cluster Quality Metrics**:
  - **Silhouette Score:** Measures how similar a point is to its own cluster vs. other clusters.
  - **Rand Index & Adjusted Rand Index**: Compare clustering results with ground truth labels.
  - **Normalized Mutual Information (NMI)**: Measures mutual dependence between clusters and true labels.

Visualization Techniques

- **Contour Plots:** Visualizing Gaussian distributions in 2D data.
- **Density Estimation:** Plotting probability density functions for each cluster.
- **Cluster Assignments:** Color-coding data points by their most probable cluster assignment.
- **Principal Component Analysis (PCA):** Reducing high-dimensional data to visualize clusters effectively.

Subsection 5

Conclusion

Summary

- GMM is a powerful clustering method using probabilistic modeling.
- EM algorithm is used for parameter estimation.
- Model selection is crucial to avoid overfitting.
- Widely used in various machine learning applications.